

Project 2: XG 四足机器人摩擦参数辨识

前置依赖: Project 1 (自由落体验证), 确认 MuJoCo 环境可用

1. 项目概述

1.1 背景: 为什么要做摩擦辨识?

在第 2 讲中我们学到: URDF/MJCF 模型中有三类参数是**缺失**的——

参数	符号	物理含义	维度
黏滞阻尼	b_i	源自润滑剂剪切, 与关节速度 $\dot{\theta}_i$ 成正比	12
库仑摩擦	$f_{c,i}$	源自齿面接触, 恒定幅值	12
转子惯量	$I_{r,i}$	电机+减速器的等效转子惯量	12

这 36 个参数在 URDF 中默认为零。仿真中"完美", 但部署到实机后, 缺失的摩擦力矩最高可占 **70%** 的关节力矩, 导致机器人直接趴下。

本项目的目标: 用可微方法从数据中辨识出 $\hat{\varphi}$, 与已知真值 φ^* 比较——**兑现主定理 (i)**。

1.2 什么是"主定理 (i)"?

课程的主定理有三条线:

- (i) 辨识 $\hat{\varphi} \leftarrow$ 本章 / 本项目
- (ii) 轨迹优化 $\leftarrow$ 第 4-5 章
- (iii) RL 策略 $\pi_{\hat{\theta}} \leftarrow$ 第 4-5 章

主定理 (i) 的工程含义: 可微物理把辨识的复杂度降到与单次仿真同阶——模型可换、梯度自动、代码不变。

1.3 分级要求

档位	辨识参数	维度	误差指标
基础	$\mathbf{b}$	12 维	$\ \hat{\mathbf{b}} - \mathbf{b}^*\ / \ \mathbf{b}^*\ < 10\%$

标准	$(\mathbf{b}, \mathbf{f}_c)$	24 维	相对误差 < 5%
进阶	全部 φ	36 维	相对误差 < 10%

1.4 交付物

- 1. Python 脚本：数据加载 + 模型注入 + loss + Adam
- 2. 三张图：损失曲线 / 梯度范数 / $\hat{\varphi}$ vs φ^*
- 3. ≤ 4 页报告

1.5 评分重点

- 不是"达到最低误差阈值"
- 而是：
- ① 解释每一步的工程权衡
 - ② 诊断收敛失败的原因
 - ③ 经典 baseline 对比是否合理

2. 环境准备

2.1 复用 Project 1 环境

代码块

```
1  conda activate freefall
```

2.2 安装额外依赖（可微辨识需要）

代码块

```
1  pip install jax jaxlib mujoco-mjx optax
```

注意：如果有 GPU，建议安装 JAX 的 CUDA 版本以加速 vmap：

代码块

```
1  pip install jax[cuda12]
```

纯 CPU 也可以运行，只是慢一些（标准档约 5-10 分钟）。

2.3 目录结构

代码块

```
1  project_2_friction_identification/
2  |—— README.md                ← 本文档
3  |—— generate_synthetic_data.py ← Step 1: 生成合成数据
4  |—— friction_identification.py ← Step 2: 辨识脚本（含两种方法）
5  |—— data_basic.npz           ← （运行 Step 1 后生成）
6  |—— data_standard.npz
7  |—— data_advanced.npz
8  |—— results_*.png            ← （运行 Step 2 后生成）
9  |—— resources/
10     |—— xg/
11         |—— xg_friction_id.xml ← XG 模型文件
```

3. 理论背景（对应 PPT 内容详解）

3.1 摩擦模型

四足机器人每个关节的摩擦力矩模型：

$$\tau_{\text{fric},i} = b_i \dot{\theta}_i + f_{c,i} \cdot \text{sgn}(\dot{\theta}_i)$$

其中 sgn 在 $\dot{\theta} = 0$ 处不可微，工程上用 $\tanh(k\dot{\theta})$ 平滑替代（ $k = 100$ ）。

3.2 带摩擦的完整运动方程

$$M(\mathbf{q})\dot{\boldsymbol{\nu}} + C\boldsymbol{\nu} + \mathbf{g} + \left[\mathbf{b} \odot \dot{\boldsymbol{\theta}} + \mathbf{f}_c \odot \tanh(k\dot{\boldsymbol{\theta}}) \right] = S^\top \boldsymbol{\tau} + \sum_i J_{c,i}^\top \mathbf{f}_c^{(i)}$$

3.3 可微辨识的损失函数

给定"实机"数据集 $\mathcal{D} = \{(\mathbf{q}_t, \dot{\mathbf{q}}_t, \dot{\boldsymbol{\nu}}_t, \boldsymbol{\tau}_t^{\text{real}})\}_{t=1}^T$ ：

$$\mathcal{L}(\varphi) = \frac{1}{T} \sum_{t=1}^T \|\boldsymbol{\tau}_t^{\text{sim}}(\varphi) - \boldsymbol{\tau}_t^{\text{real}}\|_2^2$$

辨识结果：

$$\hat{\varphi} = \arg \min_{\varphi \geq 0} \mathcal{L}(\varphi)$$

关键：梯度 $\nabla_{\varphi} \mathcal{L}$ 由 JAX 自动微分给出，不需要手工推导！

3.4 对比：经典最小二乘

经典方法把动力学方程关于 φ 显式重排为 $\boldsymbol{\tau} = W\varphi$ ，然后闭式求解：

$$\hat{\varphi}_{LS} = (W^T W + \lambda I)^{-1} W^T Y$$

维度	经典最小二乘	可微梯度方法
求解形式	闭式解	迭代优化 (Adam)
计算时间	~秒	~分钟
模型扩展	W 须重推	仅改一处仿真器代码
sgn 处理	避开 $\dot{\theta} = 0$ 区间	tanh 自然集成
$\ddot{\theta}$ 依赖	强	弱
非线性参数	不支持	任意可微模型

4. 动手实操（分步教程）

Step 1：生成合成数据

代码块

```
1 cd project_2_friction_identification
2 python generate_synthetic_data.py
```

这一步做了什么？

1. 加载 XG 模型 (`xg_friction_id.xml`), 注入 "真值" 摩擦参数 φ^*
2. 用两种激励信号跑仿真：
 - **Chirp 激励**：频率从 0.5Hz 扫到 8Hz 的正弦力矩，提供丰富频谱（满足持续激励条件）
 - **Trot 步态**：PD 控制器追踪正弦关节目标，模拟真实行走
3. 从仿真轨迹的关节速度 $\dot{\theta}$ 和已知 φ^* 直接计算摩擦力目标： $\tau_{\text{fric}} = b^* \dot{\theta} + f_c^* \text{sign}(\dot{\theta})$
4. 加 **1% 高斯噪声**（模拟传感器噪声），按 80/20 划分训练集和验证集

为什么不从 `qfric_passive` 录？MuJoCo 的 `dof_frictionloss`（库仑摩擦）通过约束求解器处理，不出现在 `qfric_passive` 字段中。因此合成数据直接从已知 计算目标，等价于 "实机采集后扣除非摩擦项" 的理想情况。

输出文件：

- `data_basic.npz`：基础档， ϕ^* 只含 `b` (12维)

- `data_standard.npz` : 标准档, ϕ^* 含 $b + f_c$ (24维)
- `data_advanced.npz` : 进阶档, ϕ^* 含 $b + f_c + l_r$ (36维)

数据格式 (以 `data_standard.npz` 为例) :

键	形状	含义
<code>dq_joint_train</code>	$(N_{\text{train}}, 12)$	关节速度
<code>ddv_joint_train</code>	$(N_{\text{train}}, 12)$	关节加速度 (中心差分)
<code>friction_target_train</code>	$(N_{\text{train}}, 12)$	摩擦力目标 (辨识的 Y)
<code>phi_star</code>	$(24,)$	真值参数

Step 2: 运行辨识

基础档 (推荐先从这里开始) :

代码块

```
1 python friction_identification.py --level basic --steps 1000
```

标准档:

代码块

```
1 python friction_identification.py --level standard --steps 2000
```

进阶档:

代码块

```
1 python friction_identification.py --level advanced --steps 3000
```

只跑最小二乘 **baseline** (不需要 JAX) :

代码块

```
1 python friction_identification.py --level standard --skip_diff
```

只跑可微辨识:

代码块

```
1 python friction_identification.py --level standard --skip_ls
```

Step 3: 查看结果

运行结束后会生成：

- 终端输出：每个参数的真值/辨识值/误差对比
- `results_{level}.png`：三合一图表
 - (a) 损失曲线（应下降 2-3 个数量级）
 - (b) 梯度范数（应趋近零）
 - (c) ϕ 与 ϕ^* 柱状图对比

5. 核心代码解读

5.1 可微摩擦模型 `friction_model`

可微辨识的核心——用 JAX 定义摩擦函数，`jax.grad` 自动求梯度：

代码块

```
1 def friction_model(phi, dq, ddv):
2     """换模型时只改这个函数，梯度由 jax.grad 自动计算。"""
3     b, fc = phi[:12], phi[12:24]
4     return b * dq + fc * jnp.tanh(100.0 * dq)
```

工程要点：

- 经典方法换模型（如 LuGre / Stribeck）须重推整个 W 矩阵
- 可微方法换模型只改这一处函数体，`jax.grad` 自动适配
- 扩展到完整仿真器时，可替换为 `mjx.step()`（整个物理引擎作为可微函数）

5.2 损失函数 `loss_fn`

代码块

```
1 def loss_fn(phi):
2     pred = jax.vmap(lambda dq, ddv: friction_model(phi, dq, ddv))(dq_jax,
3                             ddv_jax)
```

```
3     return jnp.mean((pred - target_jax) ** 2)
```

逐行解读：

1. `friction_model(phi, dq, ddv)`：用当前猜测的 ϕ 计算预测摩擦力
2. `jax.vmap`：自动向量化，一次性对 T 个时间步并行计算
3. MSE 损失：预测摩擦力 vs 实测摩擦力

5.3 梯度 & 优化

代码块

```
1 grad_fn = jax.grad(loss_fn)    # 一行代码！
2
3 for step in range(2000):
4     g = grad_fn(phi)            # 自动微分
5     updates, opt_state = opt.update(g, opt_state)
6     phi = optax.apply_updates(phi, updates)
7     phi = jnp.maximum(phi, 0.0) # 投影到非负约束
```

这就是"主定理 (i) 的工程兑现"：

- `jax.grad` 自动穿透 `friction_model` 内部所有运算，给出精确梯度
- 不需要手工推导 W 矩阵
- 换摩擦模型？只改 `friction_model` 一个函数
- 在完整工程中，`friction_model` 可替换为 `mjx.rne()` 或 `mjx.step()`，实现 PPT 中"整个仿真器作为可微函数"的完整链路

5.4 经典最小二乘 (baseline)

代码块

```
1 # 手工推导的回归矩阵 W (必须针对具体模型形式)
2 W[rows, i] = dq_joint[:, i]          # b_i 对应的列
3 W[rows, N_ACTUATOR + i] = tanh(100 * dq) # f_c, i 对应的列
4
5 # Tikhonov 正则化闭式解
6 phi_hat = solve(W'W +  $\lambda$ I, W'Y)
```

对比：换成 LuGre 模型，LS 方法的 W 必须完全重写；可微方法只改 `friction_model` 的函数体。

6. 工程权衡与常见问题

6.1 学习率选择

学习率	现象	建议
10^{-4}	收敛极慢，2000 步不够	太保守
10^{-2}	1000 步左右收敛	推荐
10^{-1}	可能振荡不收敛	太激进

6.2 收敛失败诊断

症状	可能原因	对策
Loss 不降	学习率太小 / 梯度消失	增大 lr 至 10^{-1}
Loss 振荡	学习率太大	减小 lr，或换 L-BFGS
Loss 降后回升	过拟合	检查验证集，加正则化
部分参数偏差大	数据激励不充分	检查 $\kappa(W^\top W)$ ，增加激励频段
I_r 辨识不准	缺少高频加速度数据	延长 chirp 采集时间

6.3 持续激励（PE 条件）

辨识精度的上限不仅由**算法**决定，也由**数据**决定。

b_i ：低频功率（与速度成正比）→ trot 步态即可

$f_{c,i}$ ：过零事件 → 需要速度过零，chirp 信号有帮助

$I_{r,i}$ ：高频加速度成分 → 需要丰富激励，chirp 必不可少

定量判据： $\kappa(W^\top W) < 10^4$ 表示数值稳定。

6.4 非负约束

阻尼和摩擦在物理上必须非负，因此每步优化后做投影：

代码块

```
1 phi = jnp.maximum(phi, 0.0)
```


这是投影梯度下降的标准做法。

7. 快速上手 Checklist

- ☐ 1. 确认 `conda activate freefall` 环境可用
 - ☐ 2. 安装 JAX + MJX: `pip install jax mujoco-mjx optax`
 - ☐ 3. 运行 `python generate_synthetic_data.py`，确认生成 3 个 .npz 文件
 - ☐ 4. 运行 `python friction_identification.py --level basic --steps 500` 快速验证
 - ☐ 5. 查看 results_basic.png，确认损失下降、柱状图接近
 - ☐ 6. 挑战标准档: `--level standard --steps 2000`
 - ☐ 7. 对比两种方法的误差，思考工程权衡
 - ☐ 8. 撰写 ≤ 4 页报告
-

8. 报告模板建议

1. 问题定义 (0.5 页)：什么是摩擦辨识？为什么重要？
2. 方法 (1 页)：可微方法 vs 经典方法，各自的数学公式
3. 实验 (1.5 页)：
 - 三张图（损失曲线 / 梯度范数 / ϕ vs ϕ^* ）
 - 不同档位的误差表
 - 超参数 (lr, steps) 的影响
4. 分析 (1 页)：
 - 哪些参数容易辨识？哪些难？为什么？
 - 收敛失败时的诊断过程
 - 两种方法的优劣对比